This ICCV paper is the Open Access version, provided by the Computer Vision Foundation.
Except for this watermark, it is identical to the accepted version;
the final published version of the proceedings is available on IEEE Xplore.

ZipVL: Accelerating Vision-Language Models through Dynamic Token Sparsity

Yefei He[1,2] * Feng Chen[3] Jing Liu[4] Wenqi Shao[2]
Hong Zhou[1†] Kaipeng Zhang[2†] Bohan Zhuang[1]

[1]Zhejiang University, China
[2]Shanghai AI Laboratory, China
[3]The University of Adelaide, Australia
[4]ZIP Lab, Monash University, Australia

Abstract

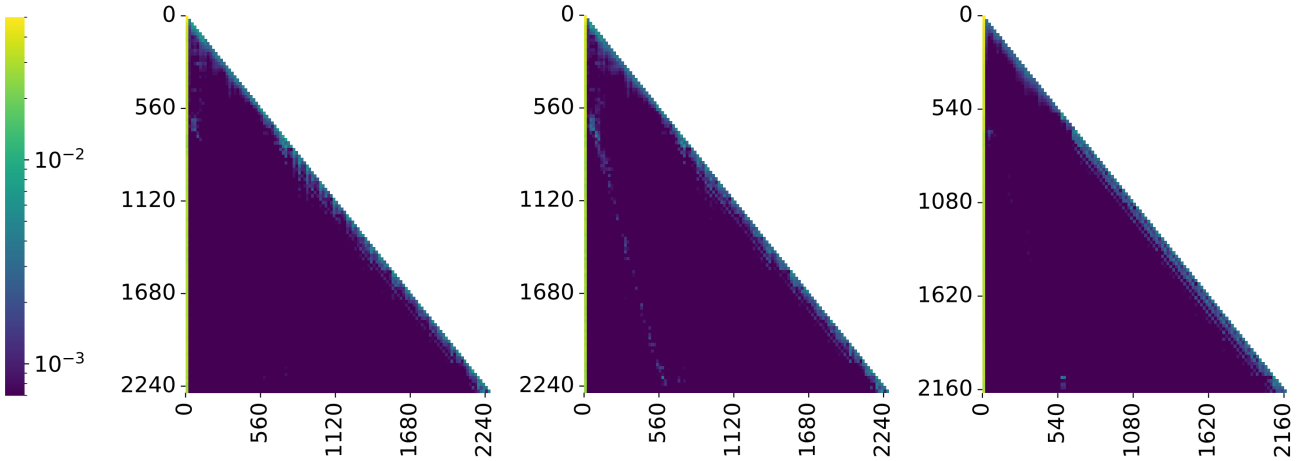
The efficiency of large vision-language models (LVLMs) is constrained by the computational bottleneck of the attention mechanism during the prefill phase and the memory bottleneck of fetching the key-value (KV) cache in the decoding phase, particularly in scenarios involving high-resolution images or videos. Visual content often exhibits substantial redundancy, resulting in highly sparse attention maps within LVLMs. This sparsity can be leveraged to accelerate attention computation or compress the KV cache through various approaches. However, most studies focus on addressing only one of these bottlenecks and do not adequately support dynamic adjustment of sparsity concerning distinct layers or tasks. In this paper, we present ZipVL, an efficient inference framework designed for LVLMs through a dynamic ratio allocation strategy of important tokens. This ratio is adaptively determined based on the layer-specific distribution of attention scores, rather than fixed hyperparameters, thereby improving efficiency for less complex tasks while maintaining high performance for more challenging ones. Then we select important tokens based on their normalized attention scores and perform sparse attention mechanism solely on those important tokens, reducing the latency in the prefill phase. Tokens deemed less important will be discarded to reduce KV cache size, alleviating the memory bottleneck in the decoding phase. Our experiments demonstrate that ZipVL can accelerate the prefill phase by 2.3× and improve decoding throughput by 2.8×, with a minimal accuracy reduction of only 0.5% on VQAv2 benchmark over LLaVA-Next-13B model, effectively enhancing the generation efficiency of LVLMs.

1. Introduction

With the recent advancement of large language models (LLMs) [1, 35, 36], many studies have extended their capabilities to comprehend and generate visual content. These models, commonly known as large vision-language models (LVLMs), have demonstrated remarkable performance in tasks such as image captioning and visual question answering [11, 12, 22, 24, 34]. Typically, to remain compatible with the next-token-prediction generation scheme of LLMs, images or videos are encoded into visual tokens through a pre-trained visual encoder, and concatenated with text tokens for input into the model. However, for high-resolution images or videos, the visual encoder generates excessive sequences of visual tokens, significantly limiting the generative efficiency of LVLMs. For example, a short video consisting of 128 frames is encoded into over 18,000 tokens by the LongVA model [43]. In such cases, the prefill phase suffers from the quadratic complexity of the attention mechanism, resulting in **computational bottleneck** and prolonged time-to-first-token (TTFT). In the decoding phase, each new token interacts with all preceding tokens, requiring to fetch the full key-value (KV) cache from memory and thereby inducing a **memory bottleneck**. Improving generative efficiency in both phases is essential for the practical deployment of LVLMs.

To address computational complexity in the prefill phase, sparse attention [18, 30, 45] has emerged as an effective strategy, particularly suitable for LVLMs where visual information exhibits considerable redundancy, resulting in higher sparsity in attention maps compared to LLMs [4, 37]. This sparsity can be implemented at various levels of granularity. Some studies pre-define several sparse patterns and assign them to the attention mask during inference [18, 45]. However, these predefined patterns are incompatible with efficient attention implementations such as FlashAttention [7] and require custom GPU kernels for each

*Work done during an internship at Shanghai AI Laboratory.
†Corresponding authors.



(a) Layer 0, ChartQA (b) Layer 10, ChartQA (c) Layer 10, VQAv2

Figure 1. The attention maps exhibit distinct sparse patterns across different layers (subfigures (a) and (b)) and vary significantly between tasks (subfigures (b) and (c)). Data was collected from the LLaVA-Next-7B model using input samples from the VQAv2 and ChartQA datasets.

pattern. Alternatively, other approaches adopt token-level sparsity by identifying and discarding less important tokens [2, 4], allowing seamless integration with off-the-shelf efficient attention implementations. However, the optimal retention ratio of important tokens may vary across different layers or tasks due to distinct attention patterns, as illustrated in Figure 1. These methods rely on a fixed token retention ratio and do not dynamically adjust based on task difficulty, leading to suboptimal performance on complex tasks.

To alleviate memory bottleneck in the decoding phase, various efforts have been made to reduce KV cache size, including token dropping [37], token merging [40], and quantization [15, 16]. However, these methods often rely on fixed compression ratios that are uniformly applied across all layers, failing to account for the distinct characteristics of attention maps in different layers. Moreover, despite the necessity of identifying important tokens for both sparse attention and KV cache compression, a unified inference optimization framework has yet to be developed.

In this paper, we present ZipVL, an efficient inference framework tailored for LVLMs that jointly optimizes the prefill and decoding phases with a unified ratio of important tokens, as shown in Figure 2. To start with, we introduce a **layer-wise adaptive ratio assignment scheme** for important tokens. This ratio is adaptively determined based on the distribution of attention scores in each layer, rather than relying on predefined hyper-parameters [2, 4, 15, 44]. This adaptive approach allows the ratio to be adjusted according to task complexity, enhancing efficiency for simpler tasks while preserving performance for more complex ones. After determining the ratio, we then select important tokens with the highest normalized attention scores, following prior work [15, 31]. To alleviate the computational bottleneck in the prefill phase, sparse attention is performed at the token level by computing attention only for the selected important tokens. Notably, this approach seamlessly integrates with existing fast attention implementations without requiring custom GPU kernels. To tackle the memory bottleneck, the same set of important tokens is applied to compress the KV cache, evicting tokens deemed less important. Extensive experiments on multimodal benchmarks demonstrate that our method achieves nearly lossless performance while reducing prefill phase latency by $2.3\times$ and improving decoding throughput by $2.8\times$.

In summary, our contributions are as follows:

- We propose an adaptive layer-wise ratio assignment scheme for important tokens. The ratio is dynamically determined based on the distribution of attention scores and varies across different layers and tasks, enhancing performance and efficiency compared to a fixed ratio scheme.
- We introduce a unified approach to jointly optimize the prefill and decoding stages through the adaptive allocation of important tokens. Tokens considered less important are excluded from attention computation to reduce computational complexity, and their KV cache will be discarded to alleviate the memory bottleneck in the decoding phase.
- By integrating these techniques, we present ZipVL, an efficient inference framework tailored for LVLMs. Comprehensive experiments across diverse benchmarks validate the efficacy of ZipVL, demonstrating that it achieves state-of-the-art performance in both accuracy and generation efficiency for LVLMs.

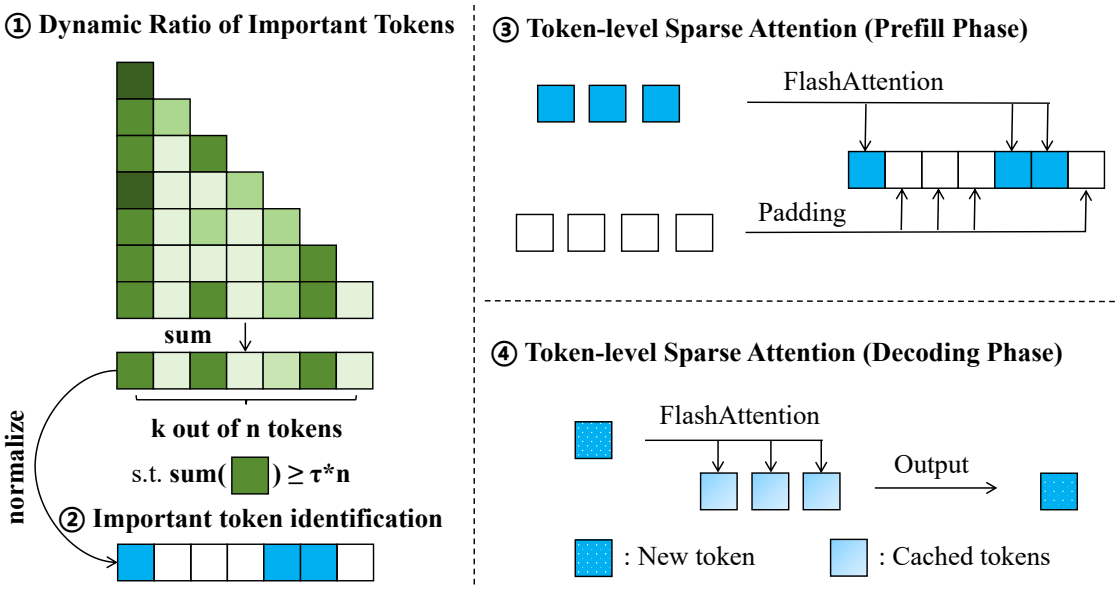


Figure 2. Overview of the proposed ZipVL framework during the prefill phase. Here, τ represents the threshold for retaining attention scores, n and p are the total number of tokens and the number of important tokens, respectively. After determining the ratio of important tokens and identifying them, we optimize both prefill and decoding phases by exclusively computing attention for important tokens. The KV cache for tokens deemed less important will be discarded.

2. Related Work

2.1. Sparse Attention

Attention scores have been widely observed to exhibit high sparsity in both LLMs and LVLMs [3, 37, 39, 42, 45]. This sparsity allows sparse attention to overcome the quadratic computational complexity of the standard attention mechanism by restricting each token to focus on only a subset of tokens within the input sequence [6, 18, 30, 32, 45]. Depending on the granularity of sparsity, sparse attention can be categorized into unstructured, semi-structured, and structured schemes. The unstructured scheme [14, 20] employs sparse attention masks without a fixed structure, making it hardware-unfriendly and challenging to achieve practical inference acceleration. The semi-structured sparse attention uses attention masks with predefined sparse patterns [18, 30, 45] or introduces N:M sparsity to attention weights [5]. However, it requires customized computational kernels for each sparse pattern or specific hardware to achieve acceleration. Structured sparse attention [2, 4] directly prunes tokens before the attention computation, enabling acceleration without the need for custom kernels. However, due to its coarse granularity, the pruning sparsity and the selection of tokens to prune significantly impact model performance. For instance, HiRED [2] selects patches with the highest responses based on the feature maps of the visual encoder without considering the input text prompt, leading to suboptimal performance. FastV [4] employs a fixed token sparsity pattern, discarding 50% of visual tokens after the second layer of the model, which results in performance degradation in challenging tasks such as ChartQA [28]. In contrast, our approach achieves superior performance by **dynamically adjusting the token sparsity ratio according to distinct models, layers, and task complexity.**

2.2. KV Cache Compression

KV cache prevents re-computation in the decoding phase by storing the key and value states of previous tokens, but with a significant memory bottleneck in long-context scenarios. Previous efforts to compress the KV cache can be broadly categorized into three types: token dropping-based [10, 31, 44], token merging-based [25, 37, 38], and quantization-based approaches [15, 16, 19, 26, 41]. In this paper, we reduce the KV cache size by dropping tokens deemed less important in a layer-wise adaptive manner. Compared to approaches with a fixed dropping ratio [4, 44], our method boosts the overall compression ratio and accuracy. Additionally, it should be noted that our method can be combined with quantization-based approaches to achieve further reductions in KV cache size.

3. Preliminary

Attention block is the key module of Transformer-based LLMs. Each attention block contains three weight matrices $\mathbf{W}_{\mathrm{Q}}, \mathbf{W}_{\mathrm{K}}, \mathbf{W}_{\mathrm{V}} \in \mathbb{R}^{d \times d}$, where d is the dimension of the input data. Here, we use a single attention head and omit the

output projection for clarity. In the prefill phase, the input data $\mathbf{X} \in \mathbb{R}^{n \times d}$ with a sequence length of n is first multiplied with three weight matrices to obtain the query, key and value states:

$$\mathbf{Q} = \mathbf{X}\mathbf{W}_{\mathrm{Q}}, \quad \mathbf{K} = \mathbf{X}\mathbf{W}_{\mathrm{K}}, \quad \mathbf{V} = \mathbf{X}\mathbf{W}_{\mathrm{V}}. \tag{1}$$

Then the attention output is calculated as follows:

$$\mathbf{A} = \mathrm{Softmax}\left(\frac{\mathbf{Q}\mathbf{K}^T + \mathbf{M}}{\sqrt{d}}\right), \mathbf{O} = \mathbf{A}\mathbf{V}. \tag{2}$$

Here, computing the product of $\mathbf{Q}\mathbf{K}^T$ has a quadratic complexity $O(n^2)$, which makes the prefill phase **compute-bound**. $\mathbf{M} \in \mathbb{R}^{n \times n}$ is a lower triangular causal mask to ensure that each token can only attend to itself and previous tokens. Unstructured and semi-structured sparse attention introduce sparsity in the attention mask $\mathbf{M}$ with dynamic or fixed sparse pattern. With custom computing kernels, tokens in certain positions can be skipped when computing $\mathbf{Q}\mathbf{K}^T$, thus accelerating the computation. On the other hand, structured sparse attention only computes attention scores for a subset of tokens $\mathbf{X}' \in \mathbb{R}^{n' \times d}$, reducing computational complexity to $O(n'^2)$ and seamlessly integrating with existing fast attention implementations.

For the decoding phase, the input data is the embedding of the current token $\mathbf{x} \in \mathbb{R}^{1 \times d}$. To enable the interaction between the current token and all previous tokens, the KV cache of previous tokens needs to be fetched from memory, making the decoding phase **memory-bound**:

$$\mathbf{q} = \mathbf{x}\mathbf{W}_{\mathrm{Q}}, \quad \mathbf{k} = \mathbf{x}\mathbf{W}_{\mathrm{K}}, \quad \mathbf{v} = \mathbf{x}\mathbf{W}_{\mathrm{V}}. \tag{3}$$

$$\mathbf{K} = \mathrm{Concat}(\mathbf{K}, \mathbf{k}), \quad \mathbf{V} = \mathrm{Concat}(\mathbf{V}, \mathbf{v}). \tag{4}$$

The attention outputs are then computed as follows with a computational complexity of $O(n)$:

$$\mathbf{a} = \mathrm{Softmax}\left(\frac{\mathbf{q}\mathbf{K}^T}{\sqrt{d}}\right), \quad \mathbf{o} = \mathbf{a}\mathbf{V}. \tag{5}$$

4. Method

4.1. Layer-wise Adaptive Ratio Assignment for Important tokens

Prior studies [2, 15, 26, 37, 44] typically adopt a fixed ratio of important tokens across all layers. However, as analyzed by the preceding study [4] and demonstrated in Figure 1(a) and (b), there are substantial variations in the attention map patterns across different layers. Moreover, Figure 1(b) and (c) illustrate that, even within the same layer, attention maps can differ depending on the task and input. In scenarios involving complex tasks, a limited, static ratio for important tokens can impair model performance. This raises the question:

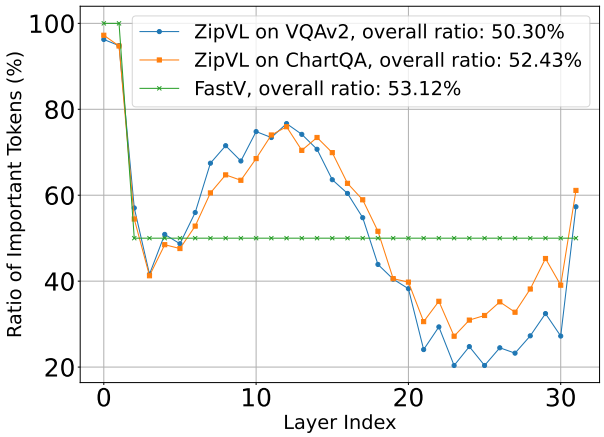


Figure 3. The ratio of important tokens distributed across layers. Data was collected from the LLaVA-Next-7B model using input samples from the VQAv2 and ChartQA datasets with a τ of 0.975.

can the model dynamically determine the number of tokens required to solve a task?

Intuitively, for simpler tasks, the model needs to concentrate on fewer tokens, leading to a more focused distribution of attention scores. Conversely, more demanding tasks require the model to engage with a broader array of tokens, resulting in a more uniform distribution of attention scores. Prior work [39] also highlights the criticality of preserving significant attention scores during inference within a constrained attention window. Building on these insights, we introduce a layer-wise adaptive scheme for assigning ratio of important tokens, ensuring the majority of significant attention scores are maintained within each layer.

Consider an attention layer with n input tokens, where the full attention score matrix is denoted as $\mathbf{A} \in \mathbb{R}^{n \times n}$. The accumulated attention score for each token j is calculated by summing the corresponding column:

$$a_j = \sum_{c=1}^{n} \mathbf{A}_{c,j}. \tag{6}$$

These accumulated attention scores are subsequently sorted in descending order, such that $a_{\mathrm{sorted}(j)}$ represents the j-th highest attention score. The number of important tokens p is determined by preserving the majority of attention scores with a minimal number of tokens, which can be expressed as:

$$p = \min\{p \in \mathbb{Z} \mid \sum_{j=1}^{p} a_{\mathrm{sorted}(j)} \geq \tau \times n\}. \tag{7}$$

Here, τ is the threshold dictating the retention of attention scores and the sum of the attention scores in $\mathbf{A}$ is equal to n due to the row-wise $\mathrm{Softmax}$ operation. As shown in Figure 2 and 3, our method can dynamically adjust the ratio

of important tokens across distinct layers and tasks, thereby enhancing performance in complex tasks while improving efficiency in simpler tasks. Additional experimental results can be found in Section 5.2.1 and Figure 4.

4.2. Inference Optimization with Unified Important Token Allocation

After determining the number of important tokens p for each layer, we partition all tokens into two sets: set $\mathbf{T}$ of important tokens with a size of p and the set $\mathbf{U}$ for less important tokens with a size of $n - p$. Following prior work [15, 31], we use normalized attention scores to assess token importance, calculated as follows:

$$\tilde{a}_j = \frac{\sum_{c=1}^{n} \mathbf{A}_{c,j}}{\text{nnz}(\mathbf{A}_{:,j})}. \tag{8}$$

Here, $\text{nnz}(\mathbf{A}_{:,j})$ denotes the number of non-zero elements in the j-th column. Important tokens are then selected using the top-k indexing method, while the remainder are considered less important:

$$\mathbf{T} = \text{topk_index}(\tilde{a}_j, p), \tag{9}$$

$$\mathbf{U} = \{j \in \{1, 2, \ldots, n\} \mid j \notin \mathbf{T}\}. \tag{10}$$

The inference optimization is then performed based on the split of tokens. Specifically, to address the **computational bottleneck** in the prefill phase, the attention mechanism is performed solely on these important tokens, thereby enhancing efficiency through token-level sparsity. Tokens excluded from this computation have their outputs padded to maintain the number of tokens consistent for subsequent layers. By leveraging token-level sparsity, our approach seamlessly integrates with off-the-shelf, fast attention implementations [7] to expedite the prefill process.

To mitigate the **memory bottleneck**, we retain the KV cache for important tokens while discarding the less important ones. During the decoding phase, the current token interacts solely with the cached important tokens, thereby reducing the memory overhead of fetching the KV cache for computation.

Efficient approximation of full attention scores. Full attention scores are not accessible in fast attention implementation [7]. In such case, to circumvent the computation of full attention scores in Eqs. (6) and (8), we selectively compute and accumulate the attention scores for a subset of tokens, following previous literature [15, 18]. The obtained partial attention scores are then used as the approximation of the full attention scores. The size of this subset is small and fixed, ensuring that the computational burden for these tokens remains minimal in long-context scenarios. The accumulated and normalized attention scores for each token can then be approximated with partial attention scores. Details and ablation experiments on this approximation can be found in the supplementary material.

Algorithm 1: The prefill phase of ZipVL

procedure `ZipVL Prefill`**:**
- **Input:** Input embedding $\mathbf{X}$, threshold τ
- **Output:** Attention output $\mathbf{O}$, KV cache $(\mathbf{K}, \mathbf{V})$
- Calculate query, key and value states ($\mathbf{Q}$,$\mathbf{K}$,$\mathbf{V}$) as per Eq. (1)
- Select a subset of tokens $\mathbf{Q}'$ from query states and compute attention scores $\mathbf{A}' = \text{Softmax}\left(\mathbf{Q}'\mathbf{K}^T\right)$
- Determine the number of important tokens as per Eq. (7)
- Calculate the normalized attention scores for each token as per Eq. (8)
- Select a set of important tokens $\mathbf{T}$ as per Eq. (9)
- `// Token-level Sparse Attention with FlashAttention`
- $\mathbf{O} = \text{FlashAttention}(\mathbf{Q}[\mathbf{T}], \mathbf{K}[\mathbf{T}], \mathbf{V}[\mathbf{T}])$
- `// KV Cache`
- $\mathbf{K} = \mathbf{K}[\mathbf{T}]$
- $\mathbf{V} = \mathbf{V}[\mathbf{T}]$
- **return** $\mathbf{O}$, $(\mathbf{K}, \mathbf{V})$

Algorithm 2: The decoding phase of ZipVL

procedure `ZipVL Decoding`**:**
- **Input:** Input embedding $\mathbf{x}$, stored KV cache $(\mathbf{K}_{\text{in}}, \mathbf{V}_{\text{in}})$
- **Output:** Attention output $\mathbf{o}$, updated KV cache $(\mathbf{K}_{\text{out}}, \mathbf{V}_{\text{out}})$
- Calculate query, key and value states ($\mathbf{q}$,$\mathbf{k}$,$\mathbf{v}$) as per Eq. (1)
- Fetch KV cache from memory: $\mathbf{K}_{\text{out}} = \text{Concat}(\mathbf{K}_{\text{in}}, \mathbf{k}), \quad \mathbf{V}_{\text{out}} = \text{Concat}(\mathbf{V}_{\text{in}}, \mathbf{v})$
- Compute attention output $\mathbf{o} = \text{FlashAttention}(\mathbf{q}, \mathbf{K}_{\text{out}}, \mathbf{V}_{\text{out}})$
- **return** $\mathbf{o}$, $(\mathbf{K}_{\text{out}}, \mathbf{V}_{\text{out}})$

Overall, the attention mechanisms for the prefill and decoding phases are summarized in Algorithms 1 and 2, respectively.

5. Experiments

5.1. Implementation Details

To assess the effectiveness of our proposed method, we conduct experiments on both image and video understanding tasks. For image understanding, we utilize widely adopted LVLMs: LLaVA [22] and LLaVA-Next [23]. These models are evaluated against five rigorous benchmarks: VQAv2 [13], TextVQA [33], GQA [17], MME [8], and ChartQA [28]. For video understanding, evaluations are

Table 1. Performance comparisons of image LVLMs on various benchmarks. Here, "Ratio" denotes the proportion of tokens participating in attention computation. "†" denotes token-level sparsity is only employed in attention modules.

Model	Method	Ratio	VQAv2	ChartQA	TextVQA	GQA	MME
LLaVA-v1.5-7B	Full	100%	76.6	18.2	46.1	61.9	1507
	FastV†	53.1%	75.8	17.7	45.5	60.2	**1511**
	HiRED	20%	73.0	17.3	45.6	56.8	1368
	HiRED	40%	75.5	17.6	45.6	59.5	1433
	Ours (τ=0.96)	44.1%	76.1	**18.0**	45.0	61.3	1495
	Ours (τ=0.975)	52.8%	**76.5**	17.9	**45.7**	**61.7**	1505
LLaVA-Next-7B	Full	100%	80.3	54.8	64.8	64.1	1519
	FastV†	53.1%	79.5	51.2	63.7	63.7	1490
	HiRED	20%	77.5	42.0	61.4	61.4	1483
	HiRED	40%	78.8	46.5	61.8	59.4	1474
	Ours (τ=0.96)	40.4%	79.4	51.0	62.8	63.8	1473
	Ours (τ=0.975)	49.3%	**79.8**	**52.6**	**64.0**	**64.0**	**1497**
LLaVA-Next-13B	Full	100%	80.9	66.2	66.9	65.7	1570
	FastV†	53.1%	76.8	51.6	59.7	62.9	1555
	HiRED	20%	77.9	48.9	63.6	63.1	1545
	HiRED	40%	79.3	53.7	**65.2**	64.1	1570
	Ours (τ=0.96)	30.6%	79.7	57.8	63.6	64.3	1551
	Ours (τ=0.975)	36.8%	**80.4**	**58.5**	64.9	**64.8**	**1574**

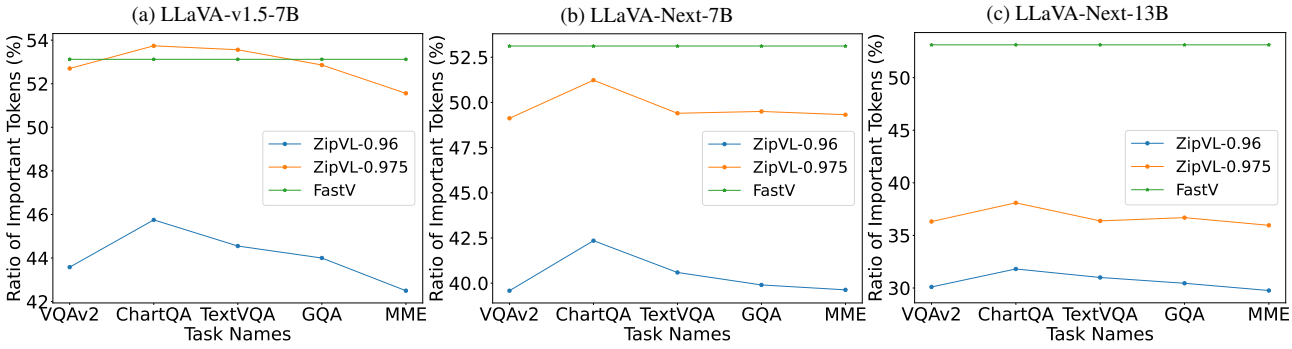


Figure 4. The ratio of important tokens across different methods on different tasks. The proposed ZipVL can adaptively determine this ratio based on the attention scores, assigning more ratio to important tokens on complex tasks.

conducted using the LongVA [43] model on the Video-MME [9] benchmark. To ensure reproducibility, all reported results are obtained using the Evaluation Suite of Large Multimodal Models [21].

5.2. Main Results

5.2.1. Evaluation on Image Benchmarks

We begin our evaluation on five image comprehension benchmarks and compare our results against well-established methods with token-level sparsity: FastV [4] and HiRED [2]. The results are presented in Table 1. Notably, HiRED determines the importance of patches through the feature map of the visual encoder, without considering the semantic information of the input prompt, resulting in a significant accuracy drop. In contrast, both FastV and our approach assess token importance via attention maps in the LVLMs. However, FastV employs a fixed token ratio and exhibits severe performance degradation on challenging tasks such as ChartQA [28]. By implementing layer-wise adaptive ratio assignment, our proposed ZipVL consistently surpasses FastV across all five benchmarks and three model architectures, while maintaining a smaller overall ratio of important tokens. As illustrated in Figure 4, our method dynamically adjusts the ratio across various tasks and models, slightly increasing the ratio of important tokens for difficult tasks to preserve performance while enhancing efficiency on simpler tasks. Moreover, the performance gap between our method and FastV becomes more pronounced over the LLaVA-Next-13B model. This discrepancy can be attributed to the varying attention maps across different models and that FastV's predefined hyperparameters are not universally applicable, whereas our dynamic approach

demonstrates high robustness.

5.2.2. Evaluation on Video Benchmarks

We also assess the performance of our method on the Video-MME benchmark [9] over the LongVA model [43], which supports a maximum multimodal input length of 224K tokens. We compare our approach with semi-structured sparse attention methods such as MInference [18] and QK-sparse [30], as well as the structured sparse attention method FastV [4]. The results are summarized in Table 2. Among these sparse attention methods, FastV [4] consistently retains a fixed proportion of tokens while MInference [18] retains a fixed number of sparse blocks. Notably, our approach not only achieves the highest overall performance but also exhibits superior reductions in FLOPs within the attention module and KV cache size compared to other sparse attention methods. This demonstrates the effectiveness of employing dynamic token-level sparsity to accelerate the attention module in LVLMs. Furthermore, long videos inherently contain significant redundancy, and our method dynamically allocates the ratio of important tokens by analyzing the sparse attention maps, resulting in a higher FLOPs reduction ratio when processing 128-frame videos compared to 64-frame videos.

Additional evaluation results on the VideoChatGPT [27] benchmark can be found in the supplementary material.

5.3. Ablation Study

5.3.1. Effect of the Layer-wise Adaptive Ratio

In this subsection, we evaluate the efficacy of the proposed adaptive ratio assignment scheme by integrating it with sparse attention and KV cache compression, as detailed in Table 3. Initially, we implement a fixed sparse attention scheme on LongVA-7B model over Video-MME benchmark. In this scheme, the ratio for important tokens remains constant across all attention layers and is fixed. Although this approach shares the same overall important token ratio and FLOPs reduction ratio as our method, it suffers from significant performance degradation (51.1% vs. 52.6%) due to its failure to account for the varying attention maps across layers. In contrast, our method achieves nearly lossless performance (52.4% vs. 52.6%) while reducing the FLOPs of attention mechanism by 82.3%.

To ablate the efficacy of our method for KV cache compression, we integrate the adaptive ratio assignment scheme with the mixed-precision KV cache quantization approach [15] and evaluate its performance over LLaMA3-8B [29] model on the GSM8k dataset. For both methods, important tokens are quantized to 4-bit, while other tokens are quantized to 2-bit. Notably, by adaptively determining the ratio of important tokens for each layer, our method achieves a significantly higher compression ratio (6.18 $\times$ vs. 4.69$\times$) while maintaining superior accuracy (54.06% vs. 53.75%). This demonstrates that our method also sets a new state-of-the-art for **KV cache compression of LLMs**.

5.3.2. Overhead Analysis

ZipVL introduces additional overhead to adaptively assign the ratio of important tokens and identify token importance. In this subsection, we provide a detailed latency breakdown analysis, as summarized in Table 4. The primary source of overhead arises from computing approximate attention scores, while other operations, such as sorting and top-k selection, contribute minimal overhead. Nonetheless, the speed gains achieved through sparse attention compensate for these costs, leading to a reduction in the overall time-to-first-token (TTFT). Furthermore, as the input sequence length increases, the impact of overhead diminishes due to the use of a fixed number of probe tokens for approximating attention scores.

Ablation experiments on the effect of τ across other models and benchmarks, as well as on probe token selection and token importance metrics, are provided in the supplementary material.

5.4. Deployment Efficiency

In this subsection, we present the prefill phase latency and decoding throughput in Table 5 to illustrate the real efficiency improvements achieved by ZipVL. Specifically, FastV [4] requires computing full attention scores to assess the token importance, which significantly increases the memory overhead. Consequently, FastV can only inference with a sequence length shorter than 16K and a small batch size. On the other hand, MInference exhibits significant additional overhead and is notably slower than FlashAttention [7] for sequence lengths below 32K. In contrast, ZipVL achieves comparable latency to FlashAttention and FastV with short input sequences, while significantly reducing the prefill phase latency as the sequence length exceeds 32K. This can be attributed to the fact that the attention module's latency becomes the dominant factor in the total latency with long sequences. With an input sequence length of 128K, ZipVL achieves a 2.3$\times$ reduction in prefill-phase latency. Moreover, MInference is not designed to reduce the KV cache size, while the proposed ZipVL jointly optimizes the attention computation and the KV cache through the dynamic token ratio. With the reduction in KV cache size, ZipVL can perform inference with a larger batch size. Consequently, ZipVL presents a 2.8$\times$ higher decoding throughput compared to baseline method [7] with an input sequence length of 16K.

6. Conclusion and Future Work

In this paper, we have proposed ZipVL, an efficient inference framework tailored for LVLMs. ZipVL jointly optimizes both the prefill and decoding phases by assigning an

Table 2. Performance comparisons of video LVLMs on Video-MME benchmark. Here, "Attn FLOPs Reduction" denotes the reduction in floating-point operations (FLOPs) of the attention mechanism. "†" denotes token-level sparsity is only employed in attention modules.

Model	Frames	Method	Attn FLOPs Reduction (%)	KV Cache Reduction (%)	Short	Medium	Long	Overall
LongVA-7B	64	Full	0	0	61.4	50.9	45.0	52.4
		QK-sparse	47.0	0	60.9	51.4	**45.1**	52.4
		MInference	54.2	0	60.7	51.2	44.6	52.1
		FastV†	71.7	46.4	61.0	50.6	45.0	52.2
		Ours(τ=0.975)	**77.0**	**52.0**	**61.0**	**51.4**	45.0	**52.4**
	128	Full	0	0	61.1	50.4	46.2	52.6
		QK-sparse	46.9	0	**61.3**	49.7	**46.3**	52.4
		MInference	77.1	0	61.0	50.5	45.3	52.3
		FastV†	71.7	46.4	60.2	50.2	46.2	52.2
		Ours(τ=0.975)	**82.3**	**57.9**	61.1	**50.5**	46.1	**52.6**

Table 3. The effect of the proposed adaptive ratio assignment scheme on sparse attention and KV cache compression. Here, "Ratio" denotes the proportion of important tokens. For Video-MME benchmark, the input videos consist of 128 frames.

Sparse Attention			
Method	Ratio (%)	Attn FLOPs Reduction (%)	Video-MME (%)
LongVA-7B	100	0	52.6
Fixed	42.1	82.3	51.1
Ours	42.1	82.3	**52.4**
KV Cache Compression			
Method	Ratio (%)	Compression Ratio	GSM8k Acc. (%)
LLaMA3-8B	100	1×	55.88
ZipCache [15]	70.0	4.69×	53.75
Ours	**28.6**	**6.18**×	**54.06**

Table 4. The overhead of each operation in ZipVL over LLaVA-Next-7B model. Here, "TTFT" denotes time-to-first-token and is measured with a batch size of 1.

Input Length	Method	TTFT (s)
16K	Original	3.48
	+ approximate attention	4.38
	+ sort & cumsum	4.41
	+ normalize & top-k	4.43
	+ sparse attention	**3.05**
32K	Original	9.35
	+ approximate attention	10.95
	+ sort & cumsum	10.96
	+ normalize & top-k	11.02
	+ sparse attention	**6.40**

Table 5. Comparisons of prefill phase latency and decoding throughput across different sequence lengths. Data is collected from LLaVA-Next-13B model on an Nvidia A100 GPU. Here, "TTFT" denotes time-to-first-token and is measured with a batch size of 1. "OOM" indicates out-of-memory during the decoding process. Throughput is measured using the maximum batch size that can be supported by a single GPU.

Input Length	Method	TTFT↓ (s)	Throughput↑ (token/s)
8K	FlashAttention	1.33	27.79
	MInference	3.30	27.79
	FastV	**1.23**	29.99
	Ours	1.31	**75.54**
16K	FlashAttention	2.64	14.10
	MInference	4.57	14.10
	FastV	OOM	OOM
	Ours	**2.61**	**40.87**
32K	FlashAttention	5.90	OOM
	MInference	6.51	OOM
	FastV	OOM	OOM
	Ours	**5.04**	**15.31**
64K	FlashAttention	15.61	OOM
	MInference	10.92	OOM
	FastV	OOM	OOM
	Ours	**10.51**	**10.29**
128K	FlashAttention	48.01	OOM
	MInference	**20.08**	OOM
	FastV	OOM	OOM
	Ours	20.57	OOM

adaptive ratio of important tokens. This ratio is dynamically adjusted based on the distribution of attention scores across each layer, ensuring that the majority of attention scores are preserved. After identifying important tokens through normalized attention scores, less significant tokens are excluded from attention computation to alleviate the computational bottleneck. Additionally, their KV cache is evicted, mitigating the memory bottleneck in the decoding phase. Extensive experiments have demonstrated that ZipVL significantly enhances the generation efficiency of LVLMs, achieving up to a 2.3× reduction in prefill phase latency and a 2.8× higher decoding throughput. However, a limitation of our approach is its focus on sparse attention only, while the multi-layer perceptron (MLP) modules in both phases remain dense. Future efforts may explore extending sparse computations to MLP modules to further reduce computational complexity.

Acknowledgement This work was supported by the National Key Research and Development Program of China (2022YFC3602601) and the National Key Research and Development Program of China (2022ZD0160102).

References

[1] Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altenschmidt, Sam Altman, Shyamal Anadkat, et al. Gpt-4 technical report. *arXiv preprint arXiv:2303.08774*, 2023. 1

[2] Kazi Hasan Ibn Arif, JinYi Yoon, Dimitrios S Nikolopoulos, Hans Vandierendonck, Deepu John, and Bo Ji. Hired: Attention-guided token dropping for efficient inference of high-resolution vision-language models in resource-constrained environments. *arXiv preprint arXiv:2408.10945*, 2024. 2, 3, 4, 6

[3] Iz Beltagy, Matthew E Peters, and Arman Cohan. Longformer: The long-document transformer. *arXiv preprint arXiv:2004.05150*, 2020. 3

[4] Liang Chen, Haozhe Zhao, Tianyu Liu, Shuai Bai, Junyang Lin, Chang Zhou, and Baobao Chang. An image is worth 1/2 tokens after layer 2: Plug-and-play inference acceleration for large vision-language models. *arXiv preprint arXiv:2403.06764*, 2024. 1, 2, 3, 4, 6, 7

[5] Zhaodong Chen, Zheng Qu, Yuying Quan, Liu Liu, Yufei Ding, and Yuan Xie. Dynamic n: M fine-grained structured sparse attention mechanism. In *PPoPP*, pages 369–379, 2023. 3

[6] Steve Dai, Hasan Genc, Rangharajan Venkatesan, and Brucek Khailany. Efficient transformer inference with statically structured sparse attention. In *2023 60th ACM/IEEE Design Automation Conference (DAC)*, pages 1–6. IEEE, 2023. 3

[7] Tri Dao, Dan Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. Flashattention: Fast and memory-efficient exact attention with io-awareness. In *NeurIPS*, 2022. 1, 5, 7

[8] Chaoyou Fu, Peixian Chen, Yunhang Shen, Yulei Qin, Mengdan Zhang, Xu Lin, Jinrui Yang, Xiawu Zheng, Ke Li, Xing Sun, Yunsheng Wu, and Rongrong Ji. Mme: A comprehensive evaluation benchmark for multimodal large language models. *arXiv preprint arXiv:2306.13394*, 2023. 5

[9] Chaoyou Fu, Yuhan Dai, Yondong Luo, Lei Li, Shuhuai Ren, Renrui Zhang, Zihan Wang, Chenyu Zhou, Yunhang Shen, Mengdan Zhang, et al. Video-mme: The first-ever comprehensive evaluation benchmark of multi-modal llms in video analysis. *arXiv preprint arXiv:2405.21075*, 2024. 6, 7

[10] Suyu Ge, Yunan Zhang, Liyuan Liu, Minjia Zhang, Jiawei Han, and Jianfeng Gao. Model tells you what to discard: Adaptive kv cache compression for llms. In *ICLR*, 2024. 3

[11] Yuying Ge, Sijie Zhao, Ziyun Zeng, Yixiao Ge, Chen Li, Xintao Wang, and Ying Shan. Making llama see and draw with seed tokenizer. In *ICLR*, 2024. 1

[12] Yuying Ge, Sijie Zhao, Jinguo Zhu, Yixiao Ge, Kun Yi, Lin Song, Chen Li, Xiaohan Ding, and Ying Shan. Seed-x: Multimodal models with unified multi-granularity comprehension and generation. *arXiv preprint arXiv:2404.14396*, 2024. 1

[13] Yash Goyal, Tejas Khot, Douglas Summers-Stay, Dhruv Batra, and Devi Parikh. Making the v in vqa matter: Elevating the role of image understanding in visual question answering. In *CVPR*, 2017. 5

[14] Team PyTorch: Horace He, Driss Guessous, Yanbo Liang, and Joy Dong. Flexattention: The flexibility of pytorch with the performance of flashattention. https://pytorch.org/blog/flexattention/, 2024. 3

[15] Yefei He, Luoming Zhang, Weijia Wu, Jing Liu, Hong Zhou, and Bohan Zhuang. Zipcache: Accurate and efficient kv cache quantization with salient token identification. In *NeurIPS*, 2024. 2, 3, 4, 5, 7, 8

[16] Coleman Hooper, Sehoon Kim, Hiva Mohammadzadeh, Michael W Mahoney, Yakun Sophia Shao, Kurt Keutzer, and Amir Gholami. Kvquant: Towards 10 million context length llm inference with kv cache quantization. *arXiv preprint arXiv:2401.18079*, 2024. 2, 3

[17] Drew A Hudson and Christopher D Manning. Gqa: A new dataset for real-world visual reasoning and compositional question answering. In *CVPR*, 2019. 5

[18] Huiqiang Jiang, YUCHENG LI, Chengruidong Zhang, Qianhui Wu, Xufang Luo, Surin Ahn, Zhenhua Han, Amir H Abdi, Dongsheng Li, Chin-Yew Lin, et al. Minference: Accelerating pre-filling for long-context llms via dynamic sparse attention. In *Workshop on Efficient Systems for Foundation Models II@ ICML*, 2024. 1, 3, 5, 7

[19] Hao Kang, Qingru Zhang, Souvik Kundu, Geonhwa Jeong, Zaoxing Liu, Tushar Krishna, and Tuo Zhao. Gear: An efficient kv cache compression recipefor near-lossless generative inference of llm. *arXiv preprint arXiv:2403.05527*, 2024. 3

[20] Benjamin Lefaudeux, Francisco Massa, Diana Liskovich, Wenhan Xiong, Vittorio Caggiano, Sean Naren, Min Xu, Jieru Hu, Marta Tintore, Susan Zhang, Patrick Labatut, Daniel Haziza, Luca Wehrstedt, Jeremy Reizenstein, and Grigory Sizov. xformers: A modular and hackable transformer modelling library. https://github.com/facebookresearch/xformers, 2022. 3

[21] Bo Li, Peiyuan Zhang, Kaichen Zhang, Fanyi Pu, Xinrun Du, Yuhao Dong, Haotian Liu, Yuanhan Zhang, Ge Zhang, Chunyuan Li, and Ziwei Liu. Lmms-eval: Accelerating the development of large multimoal models, 2024. 6

[22] Bin Lin, Bin Zhu, Yang Ye, Munan Ning, Peng Jin, and Li Yuan. Video-llava: Learning united visual representation by alignment before projection. *arXiv preprint arXiv:2311.10122*, 2023. 1, 5

[23] Haotian Liu, Chunyuan Li, Yuheng Li, Bo Li, Yuanhan Zhang, Sheng Shen, and Yong Jae Lee. Llava-next: Improved reasoning, ocr, and world knowledge, 2024. 5

[24] Haotian Liu, Chunyuan Li, Qingyang Wu, and Yong Jae Lee. Visual instruction tuning. In *NeurIPS*, 2024. 1

[25] Zuyan Liu, Benlin Liu, Jiahui Wang, Yuhao Dong, Guangyi Chen, Yongming Rao, Ranjay Krishna, and Jiwen Lu. Efficient inference of vision instruction-following models with elastic cache. *arXiv preprint arXiv:2407.18121*, 2024. 3

[26] Zirui Liu, Jiayi Yuan, Hongye Jin, Shaochen Zhong, Zhaozhuo Xu, Vladimir Braverman, Beidi Chen, and Xia Hu. Kivi: A tuning-free asymmetric 2bit quantization for kv cache. *arXiv preprint arXiv:2402.02750*, 2024. 3, 4

[27] Muhammad Maaz, Hanoona Rasheed, Salman Khan, and Fahad Shahbaz Khan. Video-chatgpt: Towards detailed video understanding via large vision and language models. In *ACL*, 2024. 7

[28] Ahmed Masry, Do Xuan Long, Jia Qing Tan, Shafiq Joty, and Enamul Hoque. Chartqa: A benchmark for question answering about charts with visual and logical reasoning. *arXiv preprint arXiv:2203.10244*, 2022. 3, 5, 6

[29] Meta. Introducing meta llama 3: The most capable openly available llm to date. https://ai.meta.com/blog/meta-llama-3/, 2024. 7

[30] Matteo Pagliardini, Daniele Paliotta, Martin Jaggi, and François Fleuret. Fast attention over long sequences with dynamic sparse flash attention. In *NeurIPS*, 2023. 1, 3, 7

[31] Siyu Ren and Kenny Q Zhu. On the efficacy of eviction policy for key-value constrained generative language model inference. *arXiv preprint arXiv:2402.06262*, 2024. 2, 3, 5

[32] Luka Ribar, Ivan Chelombiev, Luke Hudlass-Galley, Charlie Blake, Carlo Luschi, and Douglas Orr. Sparq attention: Bandwidth-efficient llm inference. In *ICML*, 2024. 3

[33] Amanpreet Singh, Vivek Natarajan, Meet Shah, Yu Jiang, Xinlei Chen, Dhruv Batra, Devi Parikh, and Marcus Rohrbach. Towards vqa models that can read. In *CVPR*, 2019. 5

[34] Chameleon Team. Chameleon: Mixed-modal early-fusion foundation models. *arXiv preprint arXiv:2405.09818*, 2024. 1

[35] Gemini Team, Rohan Anil, Sebastian Borgeaud, Yonghui Wu, Jean-Baptiste Alayrac, Jiahui Yu, Radu Soricut, Johan Schalkwyk, Andrew M Dai, Anja Hauth, et al. Gemini: a family of highly capable multimodal models. *arXiv preprint arXiv:2312.11805*, 2023. 1

[36] Raja Vavekanand and Kira Sam. Llama 3.1: An in-depth analysis of the next-generation large language model, 2024. 1

[37] Zhongwei Wan, Ziang Wu, Che Liu, Jinfa Huang, Zhihong Zhu, Peng Jin, Longyue Wang, and Li Yuan. Look-m: Look-once optimization in kv cache for efficient multimodal long-context inference. *arXiv preprint arXiv:2406.18139*, 2024. 1, 2, 3, 4

[38] Zheng Wang, Boxiao Jin, Zhongzhi Yu, and Minjia Zhang. Model tells you where to merge: Adaptive kv cache merging for llms on long-context tasks. *arXiv preprint arXiv:2407.08454*, 2024. 3

[39] Guangxuan Xiao, Yuandong Tian, Beidi Chen, Song Han, and Mike Lewis. Efficient streaming language models with attention sinks. In *ICLR*, 2024. 3, 4

[40] Dongjie Yang, XiaoDong Han, Yan Gao, Yao Hu, Shilin Zhang, and Hai Zhao. Pyramidinfer: Pyramid kv cache compression for high-throughput llm inference. *arXiv preprint arXiv:2405.12532*, 2024. 2

[41] June Yong Yang, Byeongwook Kim, Jeongin Bae, Beomseok Kwon, Gunho Park, Eunho Yang, Se Jung Kwon, and Dongsoo Lee. No token left behind: Reliable kv cache compression via importance-aware mixed precision quantization. *arXiv preprint arXiv:2402.18096*, 2024. 3

[42] Manzil Zaheer, Guru Guruganesh, Kumar Avinava Dubey, Joshua Ainslie, Chris Alberti, Santiago Ontanon, Philip Pham, Anirudh Ravula, Qifan Wang, Li Yang, et al. Big bird: Transformers for longer sequences. In *NeurIPS*, 2020. 3

[43] Peiyuan Zhang, Kaichen Zhang, Bo Li, Guangtao Zeng, Jingkang Yang, Yuanhan Zhang, Ziyue Wang, Haoran Tan, Chunyuan Li, and Ziwei Liu. Long context transfer from language to vision. *arXiv preprint arXiv:2406.16852*, 2024. 1, 6, 7

[44] Zhenyu Zhang, Ying Sheng, Tianyi Zhou, Tianlong Chen, Lianmin Zheng, Ruisi Cai, Zhao Song, Yuandong Tian, Christopher Ré, Clark Barrett, et al. H2o: Heavy-hitter oracle for efficient generative inference of large language models. In *NeurIPS*, 2023. 2, 3, 4

[45] Qianchao Zhu, Jiangfei Duan, Chang Chen, Siran Liu, Xiuhong Li, Guanyu Feng, Xin Lv, Huanqi Cao, Xiao Chuanfu, Xingcheng Zhang, et al. Near-lossless acceleration of long context llm inference with adaptive structured sparse attention. *arXiv preprint arXiv:2406.15486*, 2024. 1, 3